Тема 12. Потоки (Streams) и работа с файлами

хекслет колледж

Цель занятия:

Освоить модуль fs через призму потоковой передачи данных. Научить сервер эффективно отдавать статику и стримить медиаконтент.

хекслет колледж

Учебные вопросы:

1. Отдача статических файлов (Картинки, CSS, JS)
2. Стриминг аудио (mp3)

хекслет колледж

1. Отдача статических файлов (Картинки, CSS, JS)

Для веб-разработчика «отдача статики» - это база. Если HTML-файл запрашивает картинку или файл стилей, браузер делает отдельные HTTP-запросы на каждый из этих ресурсов.

Наша задача - научить сервер распознавать расширение файла и отдавать его с правильным «паспортом» (MIME-типом).

хекслет колледж

Когда браузер видит в HTML строку <link rel="stylesheet" href="/style.css">, он отправляет новый GET-запрос по пути /style.css.

Если наш сервер просто отдаст текст файла, но не укажет, что это CSS, браузер может проигнорировать стили из соображений безопасности

хекслет колледж

Таблица MIME-типов

Чтобы браузер правильно обработал файл, мы должны отправить заголовок Content-Type. Вот основные типы для веб-стека:

Расширение	MIME-тип
.html	text/html
.css	text/css
.js	text/javascript
.jpg / .jpeg	image/jpeg
.png	image/png

кекслет колледж

MIME-тип (Multipurpose Internet Mail Extensions) - это специальная метка (строка), которую сервер отправляет браузеру вместе с файлом, чтобы объяснить, что это за данные и как их отображать.

Представьте, что сервер отправляет браузеру посылку. Сами данные внутри - это просто набор байтов (единиц и нулей). Без «наклейки» на посылке браузер не поймет: нужно ли этот файл запустить как видео, отобразить как картинку или просто скачать на диск.

кекслет колледж

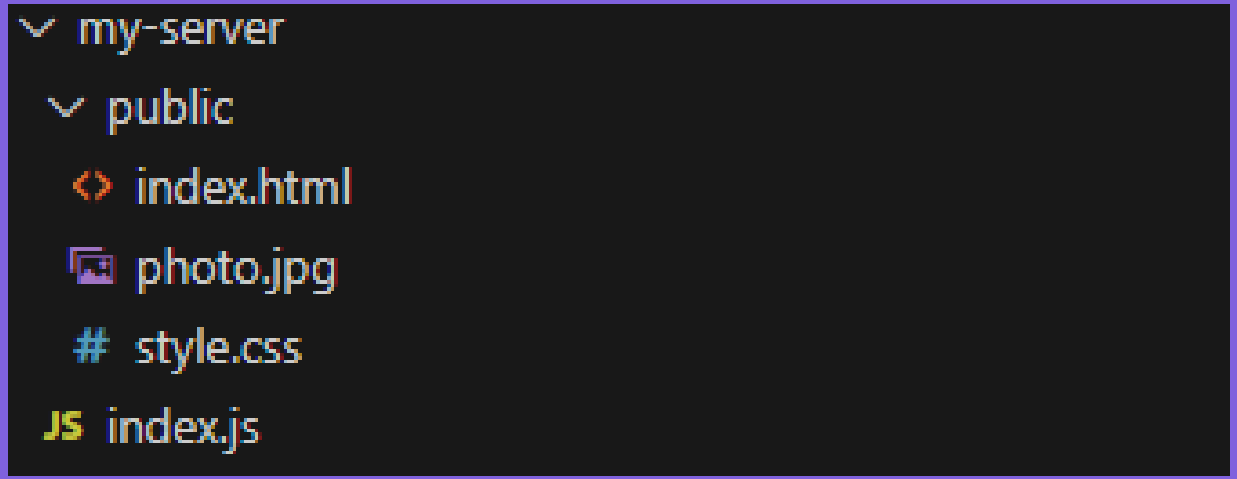
Реализация универсального обработчика

Вместо того чтобы писать if для каждого файла, мы будем извлекать расширение динамически.

хекслет колледж

Реализация:

Создадим в проекте папку public со статичными файлами:



хекслет колледж

Index.html

```html
<!doctype html>
<html>
  <head>
    <meta charset="UTF-8" />
    <link rel="stylesheet" href="style.css" />
    <title>Мой сервер статики</title>
  </head>
  <body>
    <h1>Сервер работает!</h1>
    <p>Если фон темный, а текст красный — сервер статики работает верно.</p>
    <p>Эта картинка загружена с нашего сервера:</p>
    <img src="./photo.jpg" alt="Тестовое фото" />
  </body>
</html>
```

кекслет колледж

style.css

```
body {
    background-color: #2c3e50;
    color: #ecf0f1;
    font-family: sans-serif;
    text-align: center;
}

h1 {
    color: #e74c3c;
}

img {
    max-width: 600px;
    border-radius: 8px;
    margin: 20px;
    border: 3px solid #eee;
}
```

екслет колледж

index.js

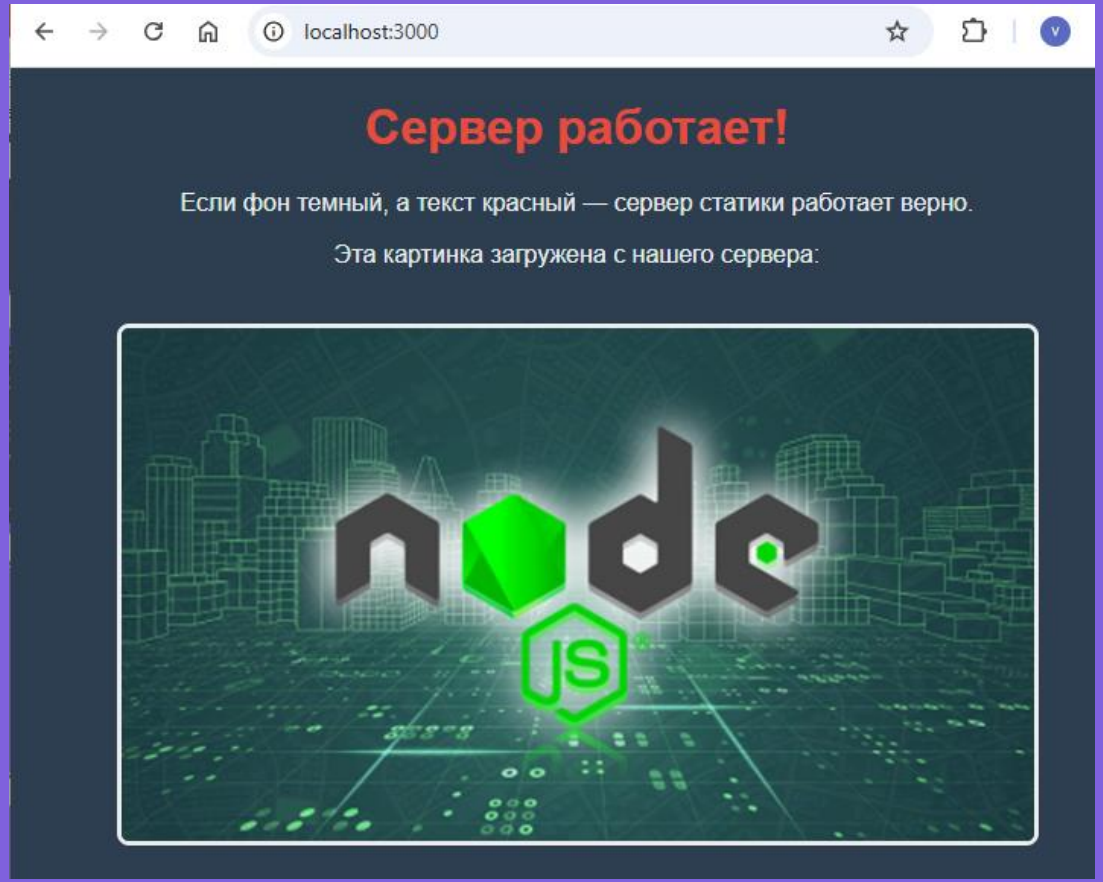
```
const fs = require('fs');
const path = require('path');
const http = require('http');

const server = http.createServer((req, res) => {
    // 1. Формируем путь к файлу в папке 'public'
    // Если зашли на '/', отдаем index.html, иначе берем имя из URL
    const fileName = req.url === '/' ? 'index.html' : req.url;
    const filePath = path.join(__dirname, 'public', fileName);

    // 2. Определяем расширение файла
    const ext = path.extname(filePath);

    // 3. Мапа MIME-типов
    const mimeTypes = {
        '.html': 'text/html',
        '.js': 'text/javascript',
        '.css': 'text/css',
        '.jpg': 'image/jpeg',
        '.png': 'image/png',
        '.ico': 'image/x-icon'
    };
```

екслет колледж

```
    // 4. Проверяем, существует ли файл
    fs.access(filePath, fs.constants.F_OK, (err) => {
        if (err) {
            res.writeHead(404, { 'Content-Type': 'text/plain; charset=utf-8' });
            res.end('404: Файл не найден');
            return;
        }

        // 5. Отдаем файл потоком (Stream) с нужным типом
        const contentType = mimeTypes[ext] || 'application/octet-stream';
        res.writeHead(200, { 'Content-Type': contentType });

        const stream = fs.createReadStream(filePath);
        stream.pipe(res);
    });
});

server.listen(3000, () => {
    console.log('Сервер статики запущен на http://localhost:3000');
});
```

кекслет колледж

Результат:



кекслет колледж

Итог:

Структура проекта: Все публичные файлы (картинки, стили) принято хранить в отдельной папке (например, public или static). Это разделяет логику сервера и ресурсы.

Безопасность: Если не ограничивать пути, пользователь может запросить ../../etc/passwd или ваш config.json с паролями. Проверка через path.join и ограничение папкой public - первый шаг к защите.

Автоматизация: Один кусок кода теперь обслуживает десятки разных картинок и скриптов без добавления новых if/else.

хекслет колледж

2. Стриминг аудио

Когда мы работаем с аудио или видео, наша цель - минимизировать время ожидания пользователя (TTFB - Time to First Byte).

Мы хотим, чтобы музыка заиграла через 100мс, а не через 5 секунд после начала загрузки.

хекслет колледж

Принцип работы потока

Когда браузер встречает тег <audio>, он делает GET-запрос к нашему серверу. Node.js не читает файл целиком.

Он создает «трубу» между жестким диском и сетевым соединением.

хекслет колледж

Почему .pipe(res) - это магия?

Метод pipe автоматически управляет скоростью передачи данных.

- Если интернет у пользователя медленный, pipe замедлит чтение с диска, чтобы не переполнять буфер.
- Если пользователь нажал на «паузу», поток данных приостановится сам собой.
- Это называется Backpressure (обратное давление) - механизм, который предотвращает «захлебывание» системы данными.

хекслет колледж

Практическая реализация

Чтобы это заработало, нам нужно добавить всего одну строку в наш список MIME-типов и закинуть любой .mp3 файл в папку public.

хекслет колледж

Чтобы это заработало, нам нужно добавить всего одну строку в наш список MIME-типов и закинуть любой .mp3 файл в папку public.

Дополнение в server.js:

```
// 3. Мапа MIME-типов
const mimeTypes = {
    '.html': 'text/html',
    '.js': 'text/javascript',
    '.css': 'text/css',
    '.jpg': 'image/jpeg',
    '.png': 'image/png',
    '.ico': 'image/x-icon',
    '.mp3': 'audio/mpeg' // добавить
};
```

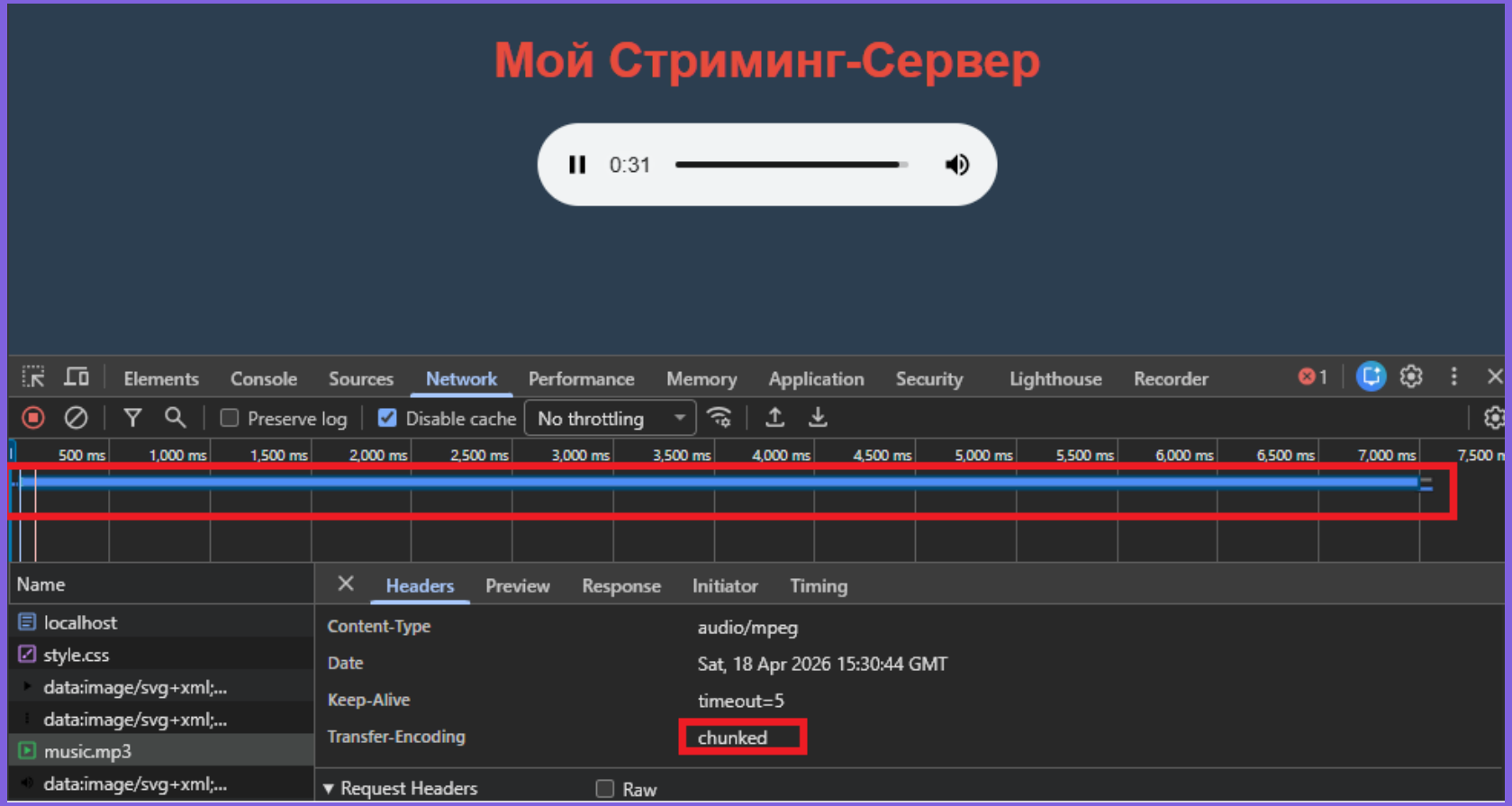
кекслет колледж

index.html

```html
<!doctype html>
<html>
  <head>
    <meta charset="UTF-8" />
    <link rel="stylesheet" href="style.css" />
    <title>Мой сервер статики</title>
  </head>
  <body>
    <h1>Мой Стриминг-Сервер</h1>
    <audio controls>
      <source src="music.mp3" type="audio/mpeg" />
    </audio>
  </body>
</html>
```

кекслет колледж

Результат:



екслет колледж

Итог:

- Стриминг - передача данных частями для мгновенного доступа.
- audio/mpeg - стандартный MIME-тип для mp3.
- Backpressure - автоматический контроль скорости потока в Node.js.
- Использование createReadStream позволяет серверу обслуживать сотни слушателей одновременно, не забивая оперативную память.

хекслет колледж

Стриминг видео в Node.js работает по тому же принципу «водопроводного крана», что и аудио, но с одним важным техническим отличием: видеофайлы обычно весят намного больше, и пользователю жизненно необходима возможность перемотки.

Если для аудио нам хватало простой передачи файла через .pipe(), то для видео браузеры ведут себя умнее: они запрашивают не весь файл целиком, а конкретные диапазоны байтов (например, «дай мне байты с 5 000 000 по 10 000 000»).

хекслет колледж

Выводы:

- Streams (Потоки) - это способ обработки данных по частям. Это база для высоконагруженных систем.
- Pipe - это "магистраль", которая соединяет источник данных и получателя, автоматически контролируя скорость (Backpressure).
- Статика - это не только HTML. Сервер должен уметь распознавать типы файлов и выдавать им правильные MIME-паспорта.

кекслет колледж

Контрольные вопросы

- В чем главная проблема использования fs.readFile для отправки видеофайла размером 2 ГБ?
- Зачем нам нужен модуль path при формировании пути к файлу?
- Что произойдет, если сервер отдаст MP3-файл, но забудет указать MIME-тип audio/mpeg?
- Как работает pipe при медленном интернете у клиента?

хекслет колледж